FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	HERNAEZ, Angelee Louise R.
Section:	AN43
Professor:	Ms. Crisola Tan
Date:	November 4, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

Source Code:

```
# PART 1: KNOWLEDGE REPRESENTATION (KR)


facts = {
    # Category 1: Temperature
    "temperature": 35,
    "feels_like": 38,


    # Category 2: Weather Conditions
```

```
    "is_raining": False,
    "humidity": 85,

    # Category 3: Weather Status
    "is_cloudy": True,
    "is_windy": False
}

print("Knowledge Base:")
for key, value in facts.items():
    print(key, ":", value)

# PART 2: RULE-BASED REASONING (RBR)

# Rule 1: Check temperature
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"

# Rule 2: Check rain
def rule_rain(facts):
    if facts["is_raining"]:
        return "Bring an umbrella"

# Rule 3: Check humidity
def rule_humidity(facts):
    if facts["humidity"] > 80:
        return "Stay hydrated"

# Rule 4: Check cloudy weather
def rule_cloudy(facts):
    if facts["is_cloudy"]:
        return "Expect possible rain"

def inference_engine(facts):
    rules = [
        rule_hot,
        rule_rain,
        rule_humidity,
        rule_cloudy
    ]

    actions = []
```

```
    for rule in rules:
        result = rule(facts)

        if result:
            actions.append(result)

    return actions

print("\nRule-Based Reasoning Actions:")

actions = inference_engine(facts)

for action in actions:
    print("-", action)

# PART 3: CASE-BASED REASONING (CBR)

case_base = [
    {
        "problem": {
            "temperature": 35,
            "is_raining": False,
            "humidity": 85
        },
        "solution": "Stay indoors and turn on the aircon."
    },

    {
        "problem": {
            "temperature": 25,
            "is_raining": True,
            "humidity": 70
        },
        "solution": "Bring an umbrella and wear a raincoat."
    },

    {
        "problem": {
            "temperature": 30,
            "is_raining": False,
            "humidity": 60
        },
```

```
        "solution": "Enjoy outdoor activities."
    }
]

def retrieve_case(new_problem, case_base):

    best_case = None
    highest_similarity = -1

    print("\nCBR Similarity Results:")

    for i, case in enumerate(case_base):

        similarity = 0

        for feature in new_problem:
            if new_problem[feature] == case["problem"][feature]:
                similarity += 1

        # Calculate similarity percentage
        percentage = (similarity / len(new_problem)) * 100

        print(f"Case {i + 1}: {percentage:.2f}% similarity")

        if percentage > highest_similarity:
            highest_similarity = percentage
            best_case = case

    print(f"\nHighest Similarity: {highest_similarity:.2f}%")

    return best_case

def reuse_solution(best_case):

    if best_case:
        return best_case["solution"]

    return "No similar case found."

def revise_solution(solution):

    print("\nCBR Suggested Solution:")
    print(solution)
```

```
    choice = input("\nDo you want to revise the solution? (yes/no): ")

    if choice.lower() == "yes":
        new_solution = input("Enter your revised solution: ")
        return new_solution

    return solution

def retain_case(new_problem, final_solution, case_base):

    new_case = {
        "problem": new_problem,
        "solution": final_solution
    }

    case_base.append(new_case)

    print("\nNew case successfully added to the case base!")

# New weather problem
new_problem = {
    "temperature": 34,
    "is_raining": False,
    "humidity": 85
}

# RETRIEVE
best_case = retrieve_case(new_problem, case_base)

# REUSE
if best_case:
    suggested_solution = reuse_solution(best_case)
else:
    suggested_solution = "No similar case found."

# REVISE
final_solution = revise_solution(suggested_solution)

# RETAIN
retain_case(new_problem, final_solution, case_base)

# Display final result
```

```
print("\nFinal CBR Solution:")
print(final_solution)

print("\nTotal Cases in Case Base:", len(case_base))
```

Output:

```
Knowledge Base:
temperature : 35
feels_like : 38
is_raining : False
humidity : 85
is_cloudy : True
is_windy : False

Rule-Based Reasoning Actions:
- Turn on the aircon
- Stay hydrated
- Expect possible rain

CBR Similarity Results:
Case 1: 66.67% similarity
Case 2: 0.00% similarity
Case 3: 33.33% similarity

Highest Similarity: 66.67%

CBR Suggested Solution:
Stay indoors and turn on the aircon.

Do you want to revise the solution? (yes/no):
```

Choice: "no" Output:

```
Do you want to revise the solution? (yes/no): no

New case successfully added to the case base!

Final CBR Solution:
Stay indoors and turn on the aircon.

Total Cases in Case Base: 4
```

Choice: "yes" Output:

```
Do you want to revise the solution? (yes/no): yes
Enter your revised solution: Stay in a cool place, use an electric fan or aircon, and drink plenty of water.

New case successfully added to the case base!

Final CBR Solution:
Stay in a cool place, use an electric fan or aircon, and drink plenty of water.

Total Cases in Case Base: 4
```

Short description of knowledge base:

The knowledge base contains weather information such as temperature, humidity, rainfall, and cloud conditions. It is used to monitor weather conditions and provide helpful recommendations, such as turning on the air conditioner, staying hydrated, or bringing an umbrella based on the current weather.

Comparative Reflection:

I found RBR easier to make because it uses simple IF-THEN rules. CBR is more flexible because it can use past experiences to solve new problems. RBR works better with changing data as long as the rules are updated, while CBR improves by adding new cases. Examples of RBR include weather alerts and medical diagnosis, while CBR is used in customer support, troubleshooting, and giving recommendations.